PHITRON AI-ML

Module 24 — Mid Term

Expense Tracker API

Topics: FastAPI • CRUD • SQLAlchemy • PostgreSQL • JWT Authentication • Authorization • Pydantic Data Validation • Routers • Query Parameters • Pytest

Total Marks: 100

Overview

You will build a Personal Expense Tracker API from scratch using FastAPI. Users will be able to register, log in, and manage their own income and expense records.

Each user should only be able to access their own transactions. The application must use a PostgreSQL database with SQLAlchemy ORM for persistent storage instead of Python lists or JSON files.

Database Models

User Table

Field	Type	Description
`id`	*int*	Primary key
`username`	*str*	Unique username
`email`	*str*	User email address
`hashed_password`	*str*	Encrypted user password

Transaction Table

Field	Type	Description
`id`	*int*	Primary key
`title`	*str*	Transaction title
`amount`	*float*	Transaction amount
`type`	*str*	"income" or "expense"
`category`	*str*	Transaction category
`date`	*date*	Transaction date
`owner_id`	*int*	Foreign key — user who created this transaction

Authentication System (20 Marks)

1. User Registration

POST	/auth/register	10 Marks

Requirements:

- Validate request data using a Pydantic model.
- Hash the password before storing it in the database.
- Save user information in the database.
- Do not return the hashed password in the response.

2. User Login

POST	/auth/login	10 Marks

Requirements:

- Verify the username and password against the database.
- Generate a JWT access token after successful authentication.
- Return the access token along with the token type.

Transaction CRUD Operations (45 Marks)

All transaction routes below must be protected using JWT authentication.

1. Create Transaction

POST	/transactions	10 Marks

Requirements:

- Automatically assign the logged-in user as the transaction owner.
- Save the transaction into the database.
- Return the created transaction.

Validation:

- amount must be a positive number.
- type must be either "income" or "expense".

2. Get All Transactions

GET	/transactions	10 Marks

Requirements:

- Return all transactions belonging to the logged-in user only.

3. Get Transaction By ID

GET	/transactions/{transaction_id}	10 Marks

Requirements:

- Return the specific transaction.
- If the transaction does not exist, return a meaningful 404 error.
- A user cannot access another user's transaction.

4. Update Transaction

PUT	`/transactions/{transaction_id}`	10 Marks

Requirements:

- A user can update only their own transaction.
- If found: apply the update and return the updated transaction.
- If not found: return a meaningful error message with status 404. Do not crash.

5. Delete Transaction

DELETE	`/transactions/{transaction_id}`	5 Marks

Requirements:

- Delete the matching row from the database.
- If found: delete it and return a confirmation message with status 200.
- If not found: return a meaningful error message with status 404. Do not crash.

Transaction Filtering (15 Marks)

Create a filtering endpoint that returns filtered transactions belonging to the logged-in user.

GET	`/transactions/filter`	15 Marks

Query Parameters:

Parameter	Example
`type`	expense
`category`	Food
`minimum_amount`	100
`maximum_amount`	5000

Example: `/transactions/filter?type=expense&category=Food`

Testing with Pytest (10 Marks)

Write 5 test cases covering the core functionality of the API:

- Get transaction test
- Get specific transaction test
- Create transaction test
- Update transaction test

- Delete transaction test

Deployment Requirement

- Upload the complete project to GitHub.
- Include a requirements.txt file listing all dependencies.
- Use PostgreSQL(online) as the database.
- Deploy the FastAPI application to a live hosting service (render).

Marking Summary

Task	Description	Marks
1	Authentication & JWT (register, login, token generation)	**20**
2	Transaction CRUD Operations	**45**
3	Database Relationship & SQLAlchemy (User ↔ Transaction, ownership checks)	**10**
4	Filtering with Query Parameters	**15**
5	Pytest (minimum 5 test cases)	**10**
	Total	**100**